

Intermediate RAGAS

Custom metrics, CI/CD eval pipelines, non-LLM metrics, scoring dashboards

01 RAGAS with Local / Custom LLM Judge

By default RAGAS uses OpenAI as the judge. For cost savings, privacy, or using your internal Qwen Coder, you can configure any OpenAI-compatible LLM as the judge.

```
# Use Ollama / internal LLM as RAGAS judge
from ragas import evaluate
from ragas.llms import LangchainLLMWrapper
from ragas.embeddings import LangchainEmbeddingsWrapper
from langchain_openai import ChatOpenAI, OpenAIEmbeddings

# Point to your internal LLM (OpenAI-compatible endpoint)
# e.g. Ollama running locally, or your Qwen Coder server
local_llm = ChatOpenAI(
    model="qwen2.5-coder:7b",
    base_url="http://localhost:11434/v1", # Ollama
    api_key="ollama")

local_embeddings = OpenAIEmbeddings(
    model="nomic-embed-text",
    base_url="http://localhost:11434/v1",
    api_key="ollama")

# Wrap for RAGAS
ragas_llm = LangchainLLMWrapper(local_llm)
ragas_emb = LangchainEmbeddingsWrapper(local_embeddings)

result = evaluate(
    dataset=dataset,
    metrics=[faithfulness, answer_relevancy],
    llm=ragas_llm,
    embeddings=ragas_emb
)
```

02 Custom Metrics — Build Your Own

```
# Custom metric: Response Conciseness
```

```

from dataclasses import dataclass, field
from ragas.metrics.base import MetricWithLLM, SingleTurnMetric
from ragas.callbacks import Callbacks

@dataclass
class ResponseConciseness(MetricWithLLM, SingleTurnMetric):
    """Measures if answer is concise without losing completeness."""
    name: str = "conciseness"

    async def _single_turn_ascore(
        self, sample, callbacks: Callbacks
    ) -> float:
        prompt = f"""Rate conciseness of this answer on scale 0-1.
        1.0 = perfectly concise (no fluff, direct answer)
        0.0 = extremely verbose with lots of irrelevant content

        Question: {sample.user_input}
        Answer: {sample.response}

        Score (just a number 0.0-1.0):"""

        result = await self.llm.generate([
            {"role": "user", "content": prompt}])
        score_text = result.generations[0][0].text.strip()
        try:
            return float(score_text)
        except ValueError:
            return 0.5 # default if LLM gives non-numeric response

# Use it
conciseness = ResponseConciseness()
result = evaluate(dataset, metrics=[conciseness, faithfulness])

```

03 Non-LLM Metrics — Fast and Free

LLM-as-judge is powerful but expensive and slow. For high-volume evaluation, use traditional metrics where possible.

```

# ROUGE, BERTScore, and retrieval metrics
from evaluate import load

# ROUGE – n-gram overlap (fast, free)
rouge = load("rouge")
result = rouge.compute(
    predictions=generated_answers,
    references=ground_truth_answers)
# {"rouge1": 0.62, "rouge2": 0.38, "rougeL": 0.55}

# BERTScore – semantic similarity (GPU recommended)
bertscore = load("bertscore")

```

```

result = bertscore.compute(
    predictions=generated_answers,
    references=ground_truth_answers,
    lang="en")
# {"precision": [...], "recall": [...], "f1": [...]}

# Hit Rate@K – did correct doc appear in top-K? (retrieval only)
def hit_rate_at_k(retrieved_ids_list, relevant_ids_list, k):
    hits = 0
    for retrieved, relevant in zip(retrieved_ids_list, relevant_ids_list):
        if any(r in retrieved[:k] for r in relevant):
            hits += 1
    return hits / len(retrieved_ids_list)

# MRR – Mean Reciprocal Rank
def mrr(retrieved_ids_list, relevant_ids_list):
    rr_sum = 0
    for retrieved, relevant in zip(retrieved_ids_list, relevant_ids_list):
        for rank, doc_id in enumerate(retrieved, 1):
            if doc_id in relevant:
                rr_sum += 1 / rank
                break
    return rr_sum / len(retrieved_ids_list)

```

04 CI/CD Evaluation Pipeline

Integrate evaluation into your CI/CD pipeline to catch regressions before deploying changes to chunking strategy, embedding model, or retrieval parameters.

```

# pytest-based eval regression test
# tests/test_rag_quality.py
import pytest
from ragas import evaluate
from ragas.metrics import faithfulness, answer_relevancy
from datasets import Dataset

# Load baseline test set (committed to repo)
import json
test_cases = json.load(open("tests/rag_test_cases.json"))

THRESHOLDS = {
    "faithfulness": 0.85,    # must be above these or test fails
    "answer_relevancy": 0.80,
}

@pytest.fixture(scope="session")
def rag_results():

```

```

"""Run RAG on all test cases once per test session."""
answers, contexts = [], []
for case in test_cases:
    result = rag_chain.invoke({"query": case["question"]})
    answers.append(result["result"])
    contexts.append([d.page_content for d in result["source_documents"]])
)

return answers, contexts

def test_faithfulness(rag_results):
    answers, contexts = rag_results
    dataset = Dataset.from_dict({
        "question": [c["question"] for c in test_cases],
        "answer": answers, "contexts": contexts})
    score = evaluate(dataset, metrics=[faithfulness])
    assert score["faithfulness"] >= THRESHOLDS["faithfulness"], \
        f"Faithfulness {score['faithfulness']:.3f} < {THRESHOLDS['faithfulness']}"

```

```

# GitHub Actions workflow for eval CI
# .github/workflows/rag-eval.yml
name: RAG Quality Check
on: [pull_request]

jobs:
  evaluate:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - uses: actions/setup-python@v4
        with: {python-version: "3.11"}
      - name: Install dependencies
        run: pip install ragas langchain pytest evaluate
      - name: Run RAG evaluation
    env:
      OPENAI_API_KEY: ${ secrets.OPENAI_API_KEY }
      QDRANT_URL: ${ secrets.QDRANT_URL }
    run: pytest tests/test_rag_quality.py -v

```

05 Evaluation Dashboard with Pandas

```

# Build a per-question score breakdown
import pandas as pd
from ragas import evaluate
from ragas.metrics import (faithfulness, answer_relevancy,

```

```

        context_precision, context_recall)

result = evaluate(dataset, metrics=[
    faithfulness, answer_relevancy,
    context_precision, context_recall])

df = result.to_pandas()

# Find worst-performing questions
df["avg_score"] = df[["faithfulness", "answer_relevancy",
    "context_precision", "context_recall"]].mean(axis=1)
worst = df.nsmallest(5, "avg_score")
print("Worst 5 questions:")
print(worst[["question", "avg_score", "faithfulness"]].to_string())

# Find retrieval failures (low context recall)
retrieval_fails = df[df["context_recall"] < 0.5]
print(f"\nRetrieval failures: {len(retrieval_fails)} questions")

# Save report
df.to_csv("eval_report.csv", index=False)

# Summary stats
print("\nSummary:")
print(df[["faithfulness", "answer_relevancy",
    "context_precision", "context_recall"]].describe())

```